6. Intention Understanding (Text Version)

1. Course Content

1. Understand what robot embodied intention understanding is, and how the large model converts fuzzy, implicit natural-language expressions from humans into precise embodied action sequences;
2. Master customizing a personalized intention mapping table through Dify's RAG (Retrieval-Augmented Generation) knowledge base;
3. Combine Nav2 autonomous navigation, deep-vision target recognition, and robotic arm gripping to realize the transition from "fuzzy intention" to "fully automatic cross-region closed-loop execution";
4. Analyze the working principles of the onboard large-model request scheduling, chain-of-thought decomposition, and the underlying action execution service.
5. **It is recommended that objects to be grasped not exceed 4 cm; the USB camera judges distance based on the size of the selected box, so it is recommended to use small block-shaped objects**

> **Important**
> **Design Intent**:
> Intention understanding is designed to enhance the interaction rapport between humans and the robot, enabling the robot to more intelligently perceive the potential needs that the user does not state directly. Please do not configure action flows that violate physical common sense or are dangerous.

---

2. Preparation

2.1 Content Description

> This lesson applies to controllers such as Raspberry Pi 5, Jetson Orin NANO, and D-Robotics RDK X5:
>
> - **Raspberry Pi 5**: You must first enter the Docker container before running ROS 2 related commands. Refer to [3.Enter Docker (For Raspberry Pi)] in the [Robot Configuration and Operation Guide];
> - **Orin / RDK X5**: Run commands directly in the onboard terminal; there is no need to enter Docker.

2.2 Start the Local Dify Large-Model Environment

Enter the following command in the host terminal to start the Dify service:

```
sh ~/bringup_dify.sh
```

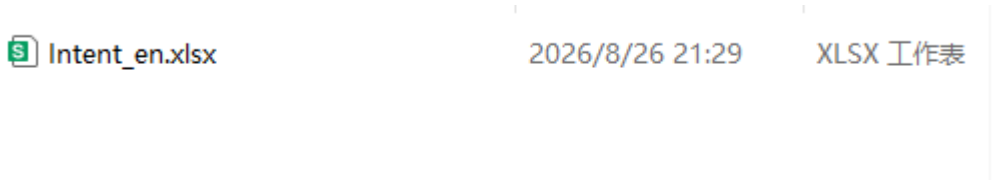
The terminal prints the following message, indicating that the Dify service is connected successfully:



2.3 Configure the Intention Mapping File (Intent mapping.xlsx)

In traditional control, you must tell the robot "go to the break room, grasp the bottled water, and bring it to the master bedroom" before it can execute. With an intention mapping table, we define the correspondence between everyday fuzzy expressions and specific embodied tasks:

Open the `Example file` folder in the same directory as this tutorial, and you will see the `Intent_en.xlsx` templates. The table contains two core columns:

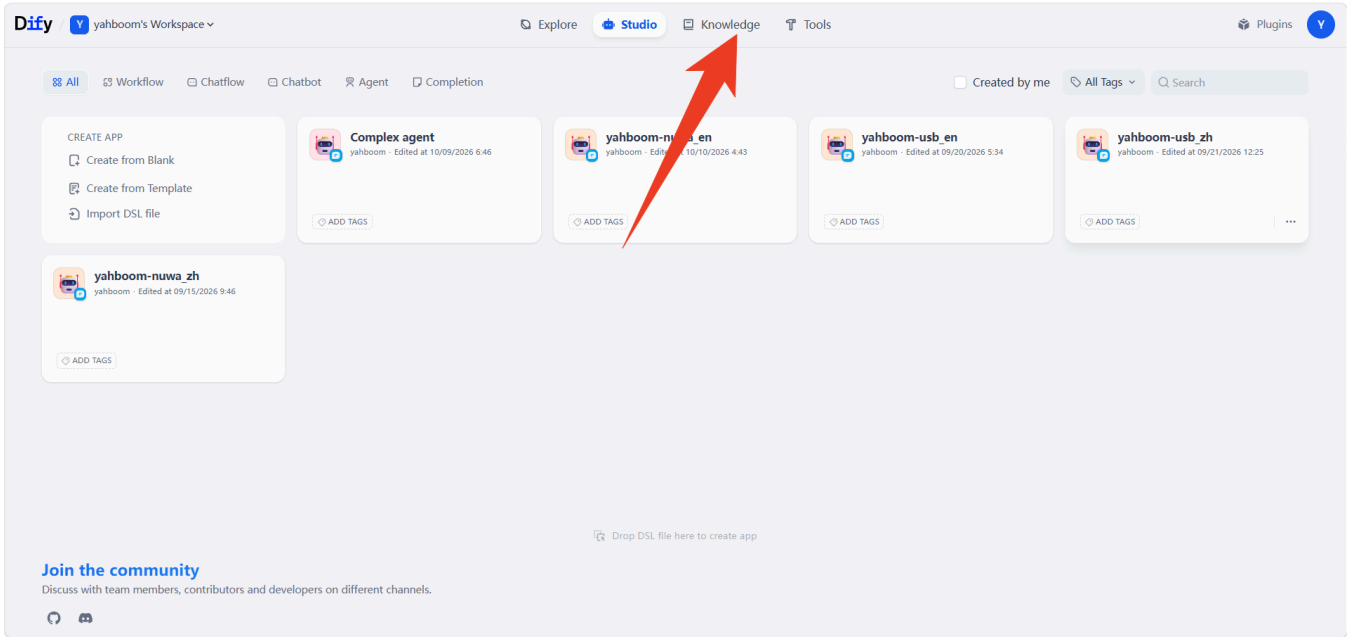


query (the user's fuzzy intention expression)	answer (the exact task steps the robot should execute)
I am in the master bedroom and I feel a little thirsty	1. Navigate to the break room 2. Find and pick up a bottle of water 3. Navigate back to the master bedroom 4. Put down the bottle of water

Users can freely expand the table with their own personalized intentions according to the actual application scenario.

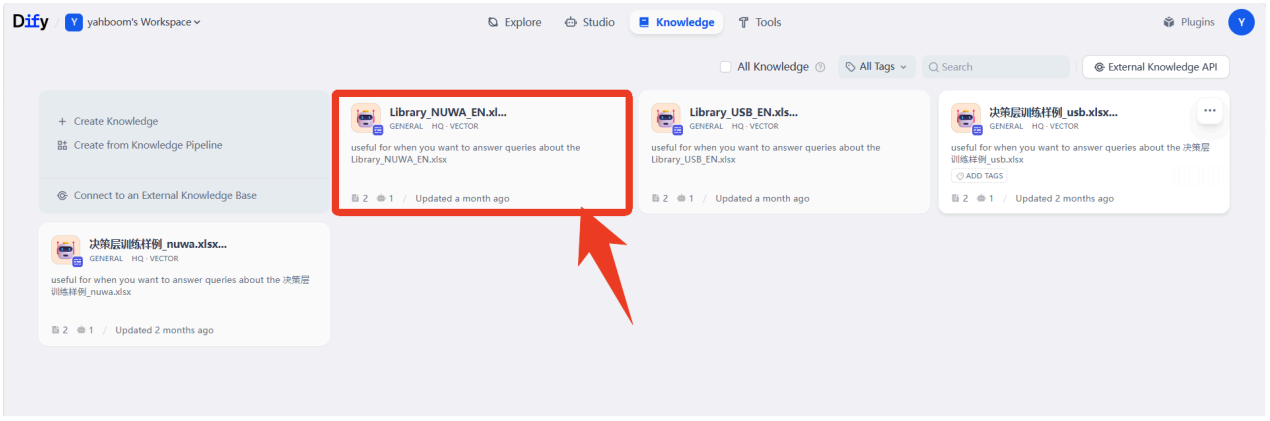
2.4 Configure the Dify RAG Knowledge Base

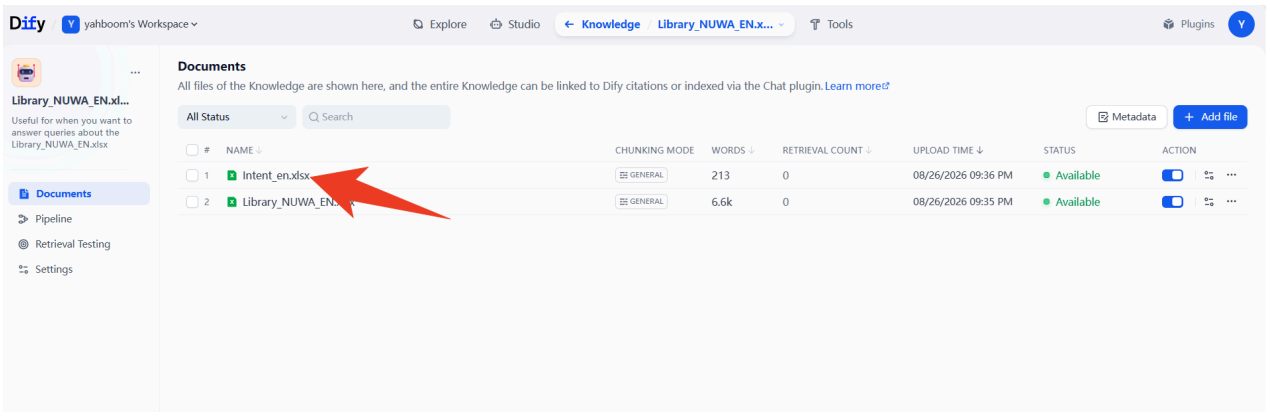
1. Access the Dify platform in a browser on a LAN computer (`<vehicle IP>`) and go to the [Knowledge] page in the top navigation bar:



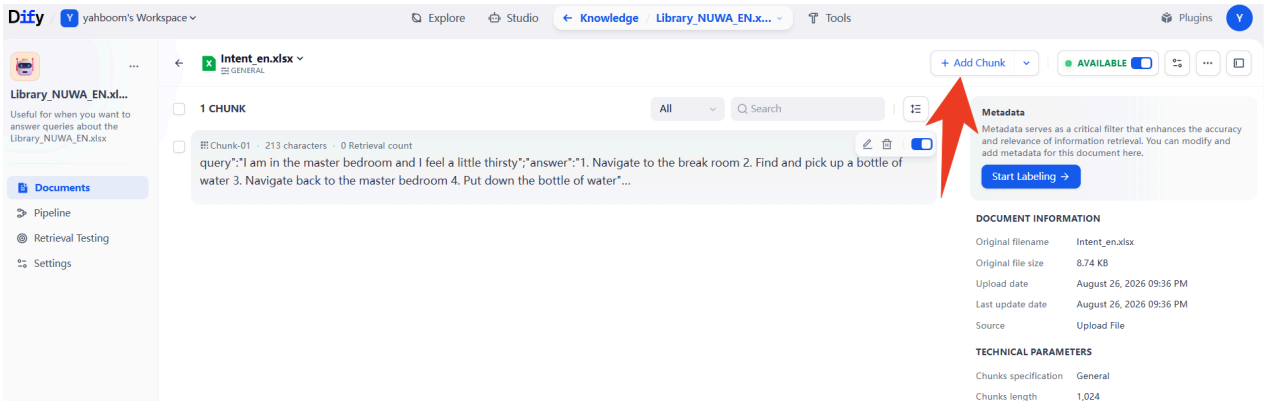
The `Intent_en` knowledge base is preconfigured at the factory.

2. Click to enter the `Intent_en` knowledge base, where you can add new entries

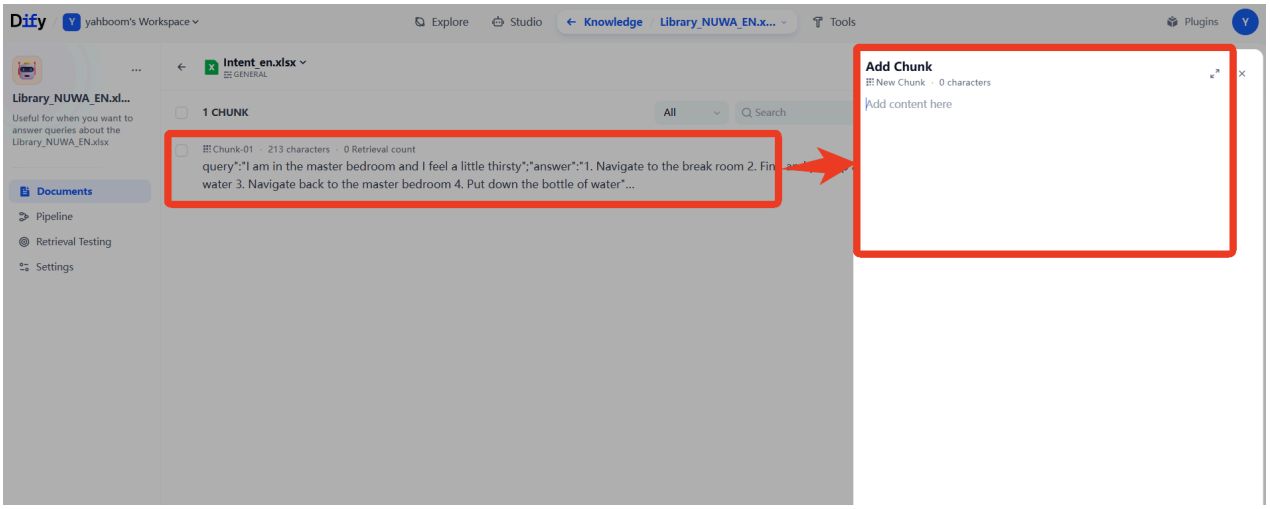




After entering, click Add Chunk,



Add your own intention inference according to the provided template



Then click Save, and then publish an update to the application.

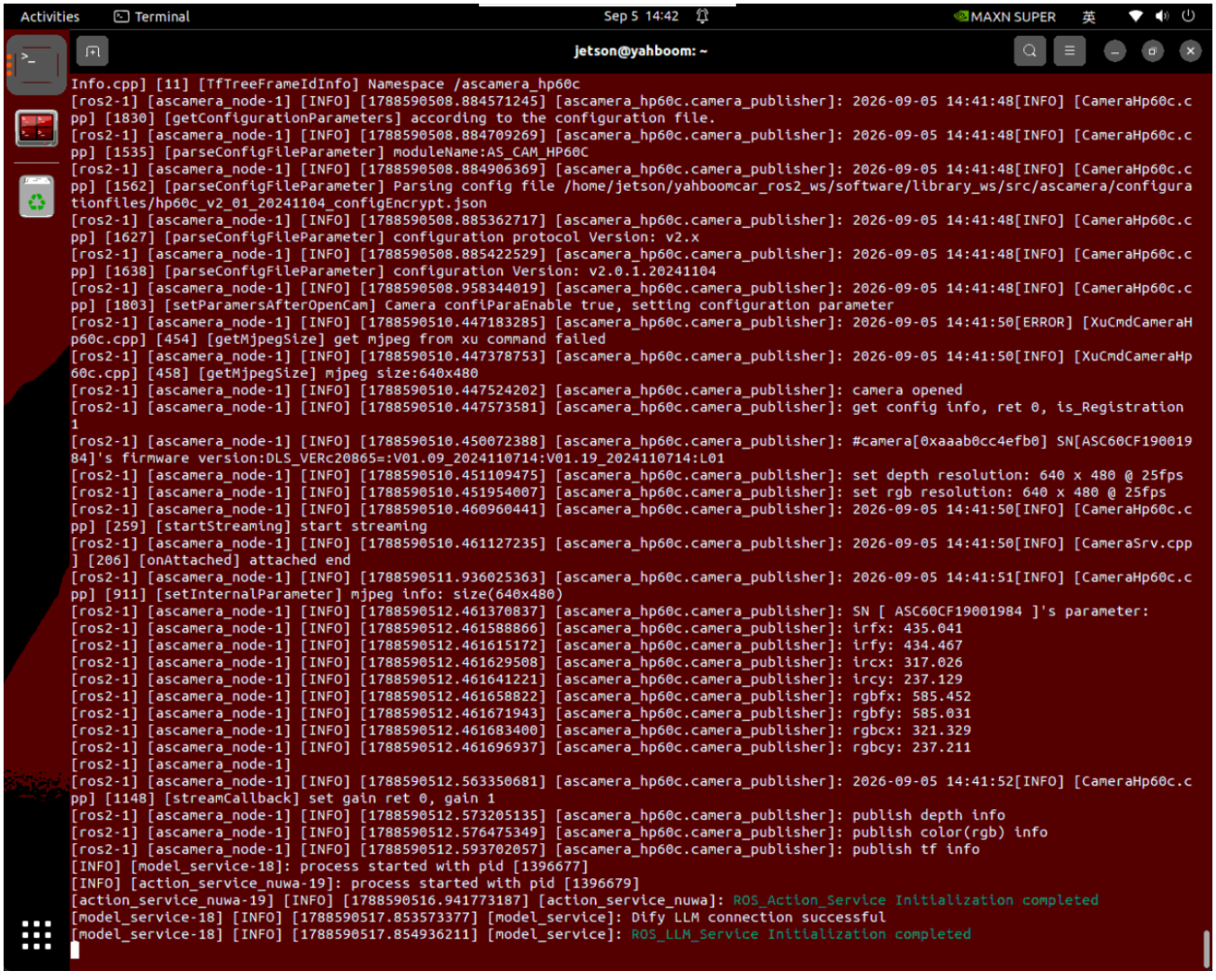
> **Tip**
>
> It is recommended to set the segment and index mode of the `Intent_en` intention understanding knowledge base to [High Quality Mode] (using vector retrieval), so that the large model can still accurately recall the corresponding task rules when the user's expression involves word-form variations or synonymous expressions.

3. Running Examples

3.1 Start the System Master Control Node (Text Interaction Mode)

Enter the following command in the onboard terminal to start the large-model embodied control service:

```
ros2 launch multi_brains llm_agent_control.launch.py text_chat_mode:=True
```



3.2 Start the SLAM Navigation and Visualization System

On the onboard desktop or a virtual machine, open another terminal and start the navigation visualization interface:

```
ros2 launch yahboomcar_nav display_nav_launch.py
```

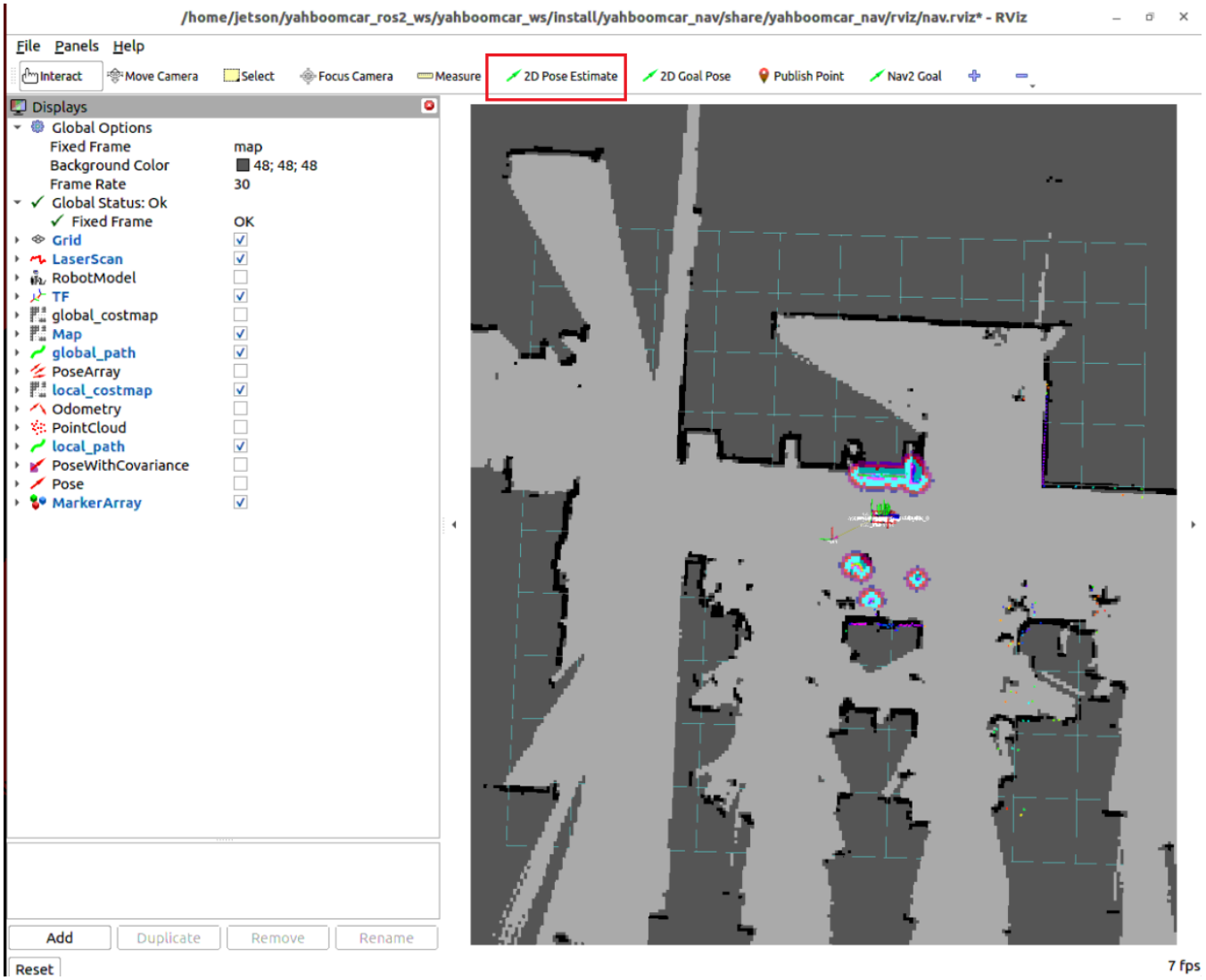
Then open another terminal and start the underlying navigation algorithm: **(the navigation algorithm must match the localization method used during waypoint calibration; if relocalization was used, run the relocalization navigation, because the poses of different localization methods may differ)**:

```
# AMCL localization + navigation
ros2 launch yahboomcar_nav navigation_teb_launch.py

# Relocalization method; see the Cartographer relocalization navigation for details
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.pbst
ream

ros2 launch yahboomcar_nav navigation_cartodwb_launch.py
maps:=$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.yaml
```

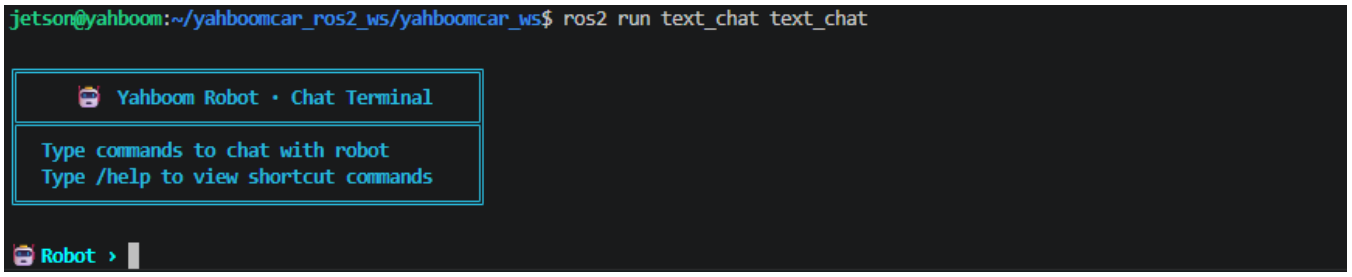
In the opened RViz2 window, use the **2D Pose Estimate** tool to perform initial alignment and localization according to the robot's parked position in the real scene.



3.3 Start the Text Interaction Terminal

Enter the following command in the SSH terminal on the computer to start the text interaction interface:

```
ros2 run text_chat text_chat
```

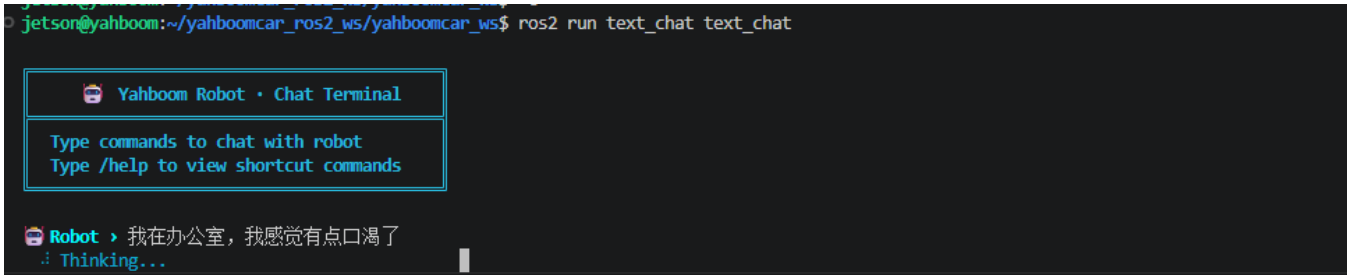


4. Execution Process and Results Display

4.1 Enter a Fuzzy Intention Command

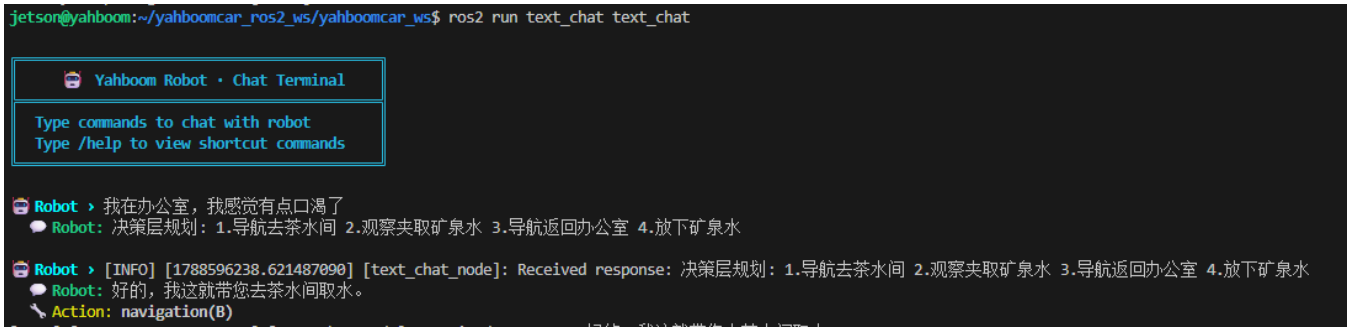
Enter everyday language in the text interaction terminal (without any explicit verb commands):

```
I am in the office, and I feel a bit thirsty.
```



4.2 Knowledge Base Recall and Task Chain-of-Thought Decomposition

After receiving the user input, the Dify agent retrieves from the RAG knowledge base and precisely matches the task template corresponding to "thirsty" from `Intent mapping`;



4.3 The Execution-Layer Large Model Schedules and Executes Item by Item

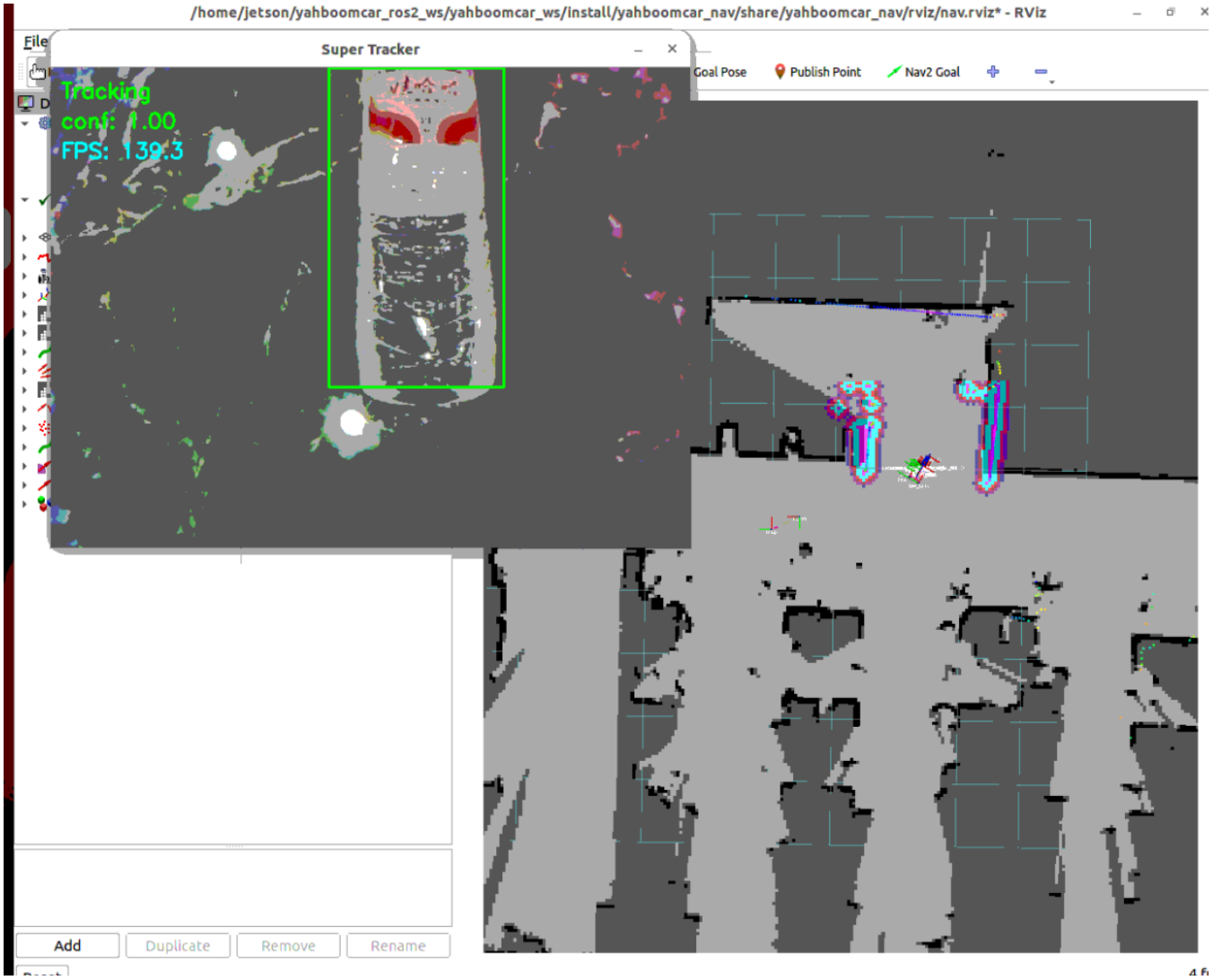
Following the above steps, the execution-layer large model issues execution requests to the lower-level action service in sequence:

```
Robot › 我在办公室，我感觉有点口渴了
  Robot: 决策层规划：1.导航去茶水间 2.观察夹取矿泉水 3.导航返回办公室 4.放下矿泉水

Robot › [INFO] [1788596238.621487090] [text_chat_node]: Received response: 决策层规划：1.导航去茶水间 2.观察夹取矿泉水 3.导航返回办公室 4.放下矿泉水
  Robot: 好的，我这就带您去茶水间取水。
  Action: navigation(B)
[INFO] [1788596238.623387454] [text_chat_node]: Received response: 好的，我这就带您去茶水间取水。
  Robot: 已经到达茶水间了，我来看看哪里有矿泉水。
  Action: ["seewhat('矿泉水')"]
[INFO] [1788596276.125518301] [text_chat_node]: Received response: 已经到达茶水间了，我来看看哪里有矿泉水。
  Robot: 看到矿泉水了，我来帮你夹取。
  Action: pick(140,20,580,220)
[INFO] [1788596285.601114581] [text_chat_node]: Received response: 看到矿泉水了，我来帮你夹取。
  Robot: 矿泉水已经拿到了，我现在带您回办公室。
  Action: navigation(A)
[INFO] [1788596319.050506050] [text_chat_node]: Received response: 矿泉水已经拿到了，我现在带您回办公室。
  Robot: 回到办公室了，我把水给您放下。
  Action: putdown()
[INFO] [1788596359.304972698] [text_chat_node]: Received response: 回到办公室了，我把水给您放下。
  Robot: 水已经给您放好了，请慢用。
[INFO] [1788596369.502933649] [text_chat_node]: Received response: 水已经给您放好了，请慢用。
```

Steps 1 & 2: Navigate to the Tea Room and Perform Visual Recognition

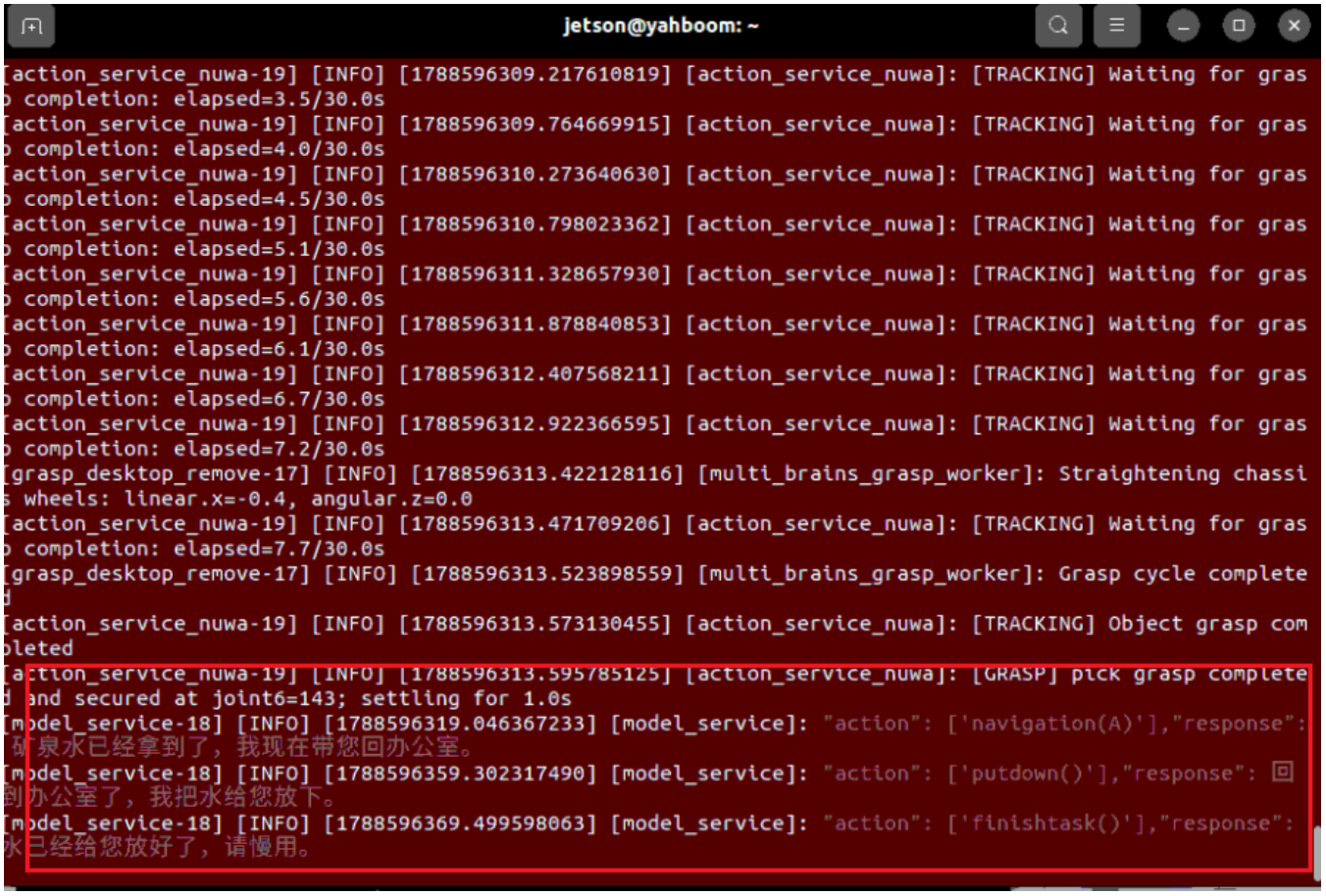
The robot chassis smoothly navigates to the "kitchen" target point, then calls the `seewhat` action to photograph the current environment, and the multimodal model locates the detection box coordinates of the target item:



> **Note**
>
> The factory default configuration of the robotic arm is suitable for grasping 30×30×30 mm building blocks or lightweight small water cups. During actual testing, items of matching size or standard test blocks can be used instead.

Steps 3 & 4: Return to the Master Bedroom, Put Down the Item, and Report

After grasping the item, the robot starts Nav2 path planning to smoothly return to the starting point "master bedroom," executes `putdown` to place the item on the table, and the text terminal outputs a completion message:



The robot now enters the waiting state; to end the current task cycle, simply enter "end the current task."

---

5. Core Source Code and Workflow Analysis

5.1 Dify Agent Intention Recognition and RAG Workflow

On the Dify platform backend, the user's input first passes through an intention routing classifier:

1. **Intent routing (Intent Classifier)**: determines whether the input is everyday chit-chat, ordinary single-step control, or a complex latent personal intention;
2. **Knowledge Retrieval**: if a latent intention category is hit, the system calls the `Intent mapping` knowledge base and uses vector similarity to retrieve the most relevant task rule text;
3. **Prompt injection and chain-of-thought generation**: injects the retrieval result into the system prompt context, guiding the large model to output the `actions` action sequence in a standard JSON format.

5.2 Onboard Action Parsing and Issuance (model_service.py)

The core onboard logic is located at `yahboomcar_ws/src/multi_brains/multi_brains/model_service.py`:

```
def extract_actions(self, llm_result: str):
```

```
    """ Extract the list of atomic actions from the result returned by the large model """
    actions = []
    # Parse the action field contained in the structured output
    ...
    return actions

def send_action_service(self, actions: list[str], text: str):
    """ Send the action list to the action execution layer through the Action client """
    if not actions:
        return
    goal_msg = LargeModelAction.Goal()
    goal_msg.actions = actions
    self.action_client.send_goal_async(goal_msg)
```

- The structured action string returned by the large model is split into standard instructions and then passed asynchronously to `ActionController` through the ROS 2 Action interface, achieving efficient cross-module scheduling.

---

6. FAQ and Notes

1. **The large model fails to recognize the intention and treats it as ordinary chit-chat**:
    - Check whether the `Intent mapping` knowledge base in the Dify platform is in the [Enabled] state, and whether the knowledge base is associated with the current Agent application;
    - Check whether the `Intent mapping.xlsx` table contains similar expressions. If the user's question differs too much from the description in the table, you can add synonymous phrasings to the knowledge base (such as "I want some water," "I'm thirsty," "get me a bottle of water").
2. **Localization is lost midway through navigation, or planning reports an error**:
    - Please make sure that the `2D Pose Estimate` initial pose calibration has been accurately completed in RViz2 before starting navigation;
    - Check whether the waypoint names of the kitchen and the master bedroom exactly match the spelling in `map_mapping.yaml`.